

PROGRAMMING LANGUAGE

USING JAVA 2

CSC 211

LECTURE NOTES

General Objectives: On completion of this course, the students should be able to:

- 1.0 Understand Array and collection processing in Java.
- 2.0 Understand Event driven programs.
- 3.0 Know the concept of inheritance, encapsulation and Polymorphism
- 4.0 Know how to use Java Servlet, and Java Server Pages (JSP)
- 5.0 Understand Database Access with JDBC 6.0 Understand the process of general enterprise solutions using Java

JavaScript is a lightweight, cross-platform, single-threaded, and interpreted compiled programming language. It is also known as the scripting language for webpages. It is well-known for the development of web pages, and many non-browser environments also use it.

Arrays: An array represents a group of elements of same data type. Array is group of elements of similar types occupying contiguous memory locations.

Arrays of Primitive Data Types

An array is a collection of similar data types. Array is a container object that hold values of homogenous type. It is also known as static data structure because size of an array must be specified at the time of its declaration.

An array can be either primitive or reference type.

Array of objects statement

An array in java script is the collection of multiple types of data at a single place in order, and an array of objects in java script is a collection of homogenous data, that stores a sequence of numbered objects at a single place.

Example-

```
let my arr = [  
  {  
    Student Name: " jack ",  
    Student Class: 6,  
    Student Section: " a "  
  },  
  {  
    Student Name: " tom ",  
    Student Class: 6,  
    Student Section: " a "  
  }  
];
```

```
console. Log (my arr [ 1]);
```

Output-

```
{
```

```
Student Name: " tom ",  
Student Class: 6,  
studenSection: "a"  
}
```

Object Manipulation in JavaScript

What is an Object?

An object is a collection of properties, and a property is an association between a key and a value. A property's value can be a number, string, Boolean, null, undefined, object, or function.

An object has three characteristics:

1. **State:** represents the data (value) of an object.
2. **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
3. **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.

Object can be created, assign it to a variable, add properties to it, and delete properties from it, and we can also access the values of the properties using the dot notation or the bracket notation.

The storage and retrieval process of a 1- dimensional and 2- dimensional array

A One-Dimensional Array is a special type of variable that can store multiple values of only a single data type such as int, float, double, char, etc. at a contiguous location in computer memory. Here contiguous location means at a fixed gap in computer memory. A One-Dimensional Array is also known as 1D Array.

Suppose we want to store the age of 10 students. In that case, we have to declare 10 variables in our Java program to store the age of 10 students.

Now here comes the use of a one-dimensional array. With the help of 1D Array, we will declare a single variable in our Java program that can store the age of 10 students at a time.

Declaration Syntax of a One-Dimensional Array in Java

Data type variable _ name [] = new data type [size];

Or

Data type [] variable name = new data type [size];

Here, size is the number of elements we want to store in the array.

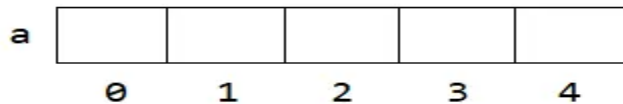
Example

```
int a [ ] = new int [ 5 ] ;
```

Or

```
Int [ ] a = new int [5];
```

Once we declare the 1D Array, it will look like as shown in the picture below:



Size of the array is 5.

Index of the array is 0, 1, 2, 3 and 4.

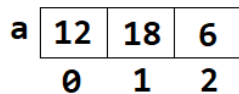
The first index is called Lower Bound, and the last index is called an Upper Bound.

Declaration and Initialization of a One-Dimensional Array in Java

In Java programming a one-dimensional array can be declared and initialized in several ways. Let's see the different ways of initializing a 1D array.

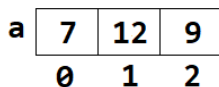
Example 1:

```
int a [ ] = new int [ ] { 12, 18, 6 } ;
```



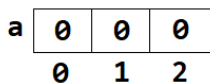
Example 2:

```
int a [ ] = { 7, 12, 9 } ;
```



Example 3:

```
int a [ ] = new int [ 3 ] ;
```



Note: If an array is initialized, without assigning any value, then the default value of each cell of the array will be 0.

Storing Numbers in a One-Dimensional Array

To store the number in each cell of the array we can use the following syntax.

```
Array _ name [index] = value;
```

Example

```
a [0] = 26;
```

```
a [1] = 15;
```

```
a [2] = 34;
```

Accessing Numbers in a One-Dimensional Array

We can access any number stored in a 1D array using the following syntax.

```
Array _ name [index];
```

Example

```
System. out. Println (a [0] + " " + a [1] + "" + a [2] );
```

Output

```
26 15 34
```

The storage and retrieval process of a 2- dimensional array (2D Array)

A Two-Dimensional Array in Java is a collection of 1D Array. It consists of rows and columns and looks like a table. A 2D array is also known as Matrix.

Declaration Syntax of a Two-Dimensional Array in Java

```
Data type variable _ name [ ] [ ] = new data type [row _ size] [column size];
```

Or

```
Data type [ ] [ ] variable _ name = new data type [row _ size] [column _ size];
```

Here row _ size is the number of rows we want to create in a 2D array and, column _ size is the number of columns in each row.

Example

```
int a [ ] [ ] = new int [ 3 ] [ 3 ] ;
```

Or

```
Int [ ] [ ] a = new int [3] [3];
```

Once we declare the 2D Array, it will look like as shown in the picture below:

		Column	Column	Column
		0	1	2
Row	0	0, 0	0, 1	0, 2
Row	1	1, 0	1, 1	1, 2
Row	2	2, 0	2, 1	2, 2

In the above image, you can see that we have created a 2D Array having 3 rows and 3 columns. We can call the above array as a 3x3 Matrix.

The first row and column always start with index 0.

Note: A 2D array is used to store data in the form of a table.

Declaration and Initialization of a Two-Dimensional Array in Java

In Java programming a two-dimensional array can be declared and initialized in several ways. Let's see the different ways of initializing a 2D array.

Example 1:

```
int a [ ] [ ] = new int [ ] [ ] { { 1, 2, 3 } , { 4, 5, 6 } , { 7, 8, 9 } } ;
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Example 2:

```
int a [ ] [ ] = { { 15, 27, 36 } , { 41, 52, 64 } , { 79, 87, 93 } } ;
```

	0	1	2
0	15	27	36
1	41	52	64
2	79	87	93

Example 3: (All cell contains the value 0)

```
int a [ ] [ ] = new int [ 3 ] [ 3 ] ;
```

	0	1	2
0	0	0	0
1	0	0	0
2	0	0	0

Storing Numbers in a Two-Dimensional Array

To store the number in each cell of the 2D array we can use the following syntax.

```
Array _ name [row _ index] [column_ index] = value;
```

Example

```
a [ 0 ] [ 0 ] = 15;
```

```
a [ 0 ] [ 1 ] = 27;
```

```
a [ 0 ] [ 2 ] = 36;
```

```
a [ 1 ] [ 0 ] = 41;
```

```
a [ 1 ] [ 1 ] = 52;
```

```
a [ 1 ] [ 2 ] = 64;
```

```
a [ 2 ] [ 0 ] = 79;
```

```
a [ 2 ] [ 1 ] = 87;
```

```
a [ 2 ] [ 2 ] = 93;
```

```
(" \n Output “) ;
```

```
For (r = 0; r < 3; r ++ ) // this loop is for row
```

```
{Access Numbers in a Two-Dimensional Array
```

What are Stacks and Queues?

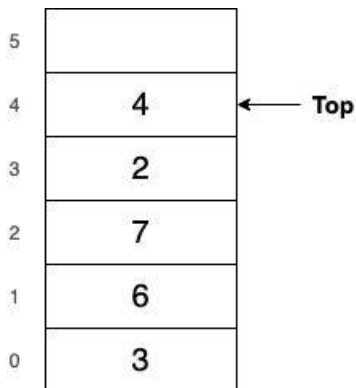
Stacks and Queues are usually explained together because they are similar data structures. Both are linear data structures. The difference between them is how elements are removed.

What is a Stack?

A Stack is a LIFO (Last In First Out) data structure. The last element that is placed in a stack is the first element that can be removed. Elements can be inserted and deleted only from one side of the list, called the top.

Stacks are used in a variety of ways when coding. Stacks are used to implement functions, parsers, expression evaluation, and some algorithms. Stacks are great for processing nested structures, so they are important for understanding *recursion*.

The insertion of an element into stack is called pushing. Deletion of an element from the stack is called popping. In stacks, the last element in a list is tracked with a pointer called top.



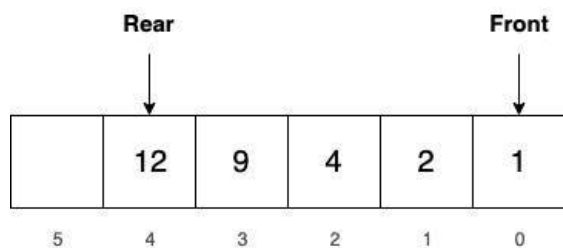
What is a Queue?

A Queue is a FIFO (First In First Out) data structure. The first element that is placed in a queue is the first one out. Elements can be inserted only from one side of the list called **rear**, and the elements can be deleted only from the other side called the front. Think of queues like a queue of people waiting for something. The first person to queue up is the first person served.

The insertion of an element in a queue is called an enqueue operation and deleting an element is called a dequeue operation. For queues, two pointers are maintained;

Front pointer: points to the element which was inserted at the first and still present in the list.

Rear pointer: points to the element inserted last.



Types of Queues

There are common types of queues you can expect to encounter.

Circular Queue: In a circular queue, the last element is connected to the first element to form a circle.

Priority Queue: In priority queues, elements are sorted based on priority.

Stack and Queue Implementation

How do Stacks work?

The implementation of stacks is relatively easy. The functionality depends on the pop and push method, as you can see from the illustration above. The pop method **removes** or deletes elements from the stack, while the push method inserts or adds items to the stack.

When an element is inserted into a stack, it takes the top position and the variable storing this position points to the number below it. The top variable should be updated anytime an element is inserted or removed from it.

Note: What's important to remember is that insertion and deletion happen on the same end of a Stack.

Vectors in JavaScript

A vector is a special kind of array whose size can be increased or decreased based on the data stored in it. Vectors can hold any type of elements, objects and data types. It can be said that vectors are one-dimensional dynamic arrays. Due to its dynamic structure, it provides the ability to assign the structure size before-hand and enable memory space allocation quickly. Now we have defined what vectors are let's understand why they are used.

Why Vectors are used in JavaScript?

The points stated below explain to us why vectors are used even though we have arrays, and what benefits/features it provides that makes it useful in programming:

1. Vectors can be used to store objects or a collection of objects in an organized fashion.
2. The size of the vector can be provided upfront.
3. Serves as a data structure container.
4. It provides memory space allocation quickly.
5. Dynamic in nature i.e. their size can be increased on-demand, unlike arrays, that have a fixed size.

6. Accessing an item inside vector via an index in constant time, just like an array.

Examples of Vector in JavaScript

Let's see how a vector can be created in JavaScript and its properties by going through some examples:

```
<!DOCTYPE HTML>

<Html >

<Body > </body >

<Script >

// Created new Vector – num Books

var num Books = ["first Book", "second Book", "third Book"];

for (var count = 0; count < num Books. length; count++)

console. Log ("Printing Books - “, count,": ", num Books [ count ] ) ;

</ script >

</ html >
```

2.1 Placing buttons on a Frame

In Java, a frame is a window that has nice borders, various buttons along the top border, and other features. What you usually call a "window" Java calls a "frame". A frame is a container object, so GUI components can be placed in it.

Like all software objects, a frame object is actually a section of main memory that holds information and methods. With the help of the operating system and the graphics board, Java paints a picture on the computer monitor that represents the frame. People often speak as if the frame were the actual picture on the monitor. In fact, what you see on the monitor is just a graphical representation of the frame. GUI application programs are usually organized around one or more frames.

In Java, you can create clickable buttons using the JButton class, which is part of the Swing library bundled with the Java Development Kit (JDK). These buttons can serve various purposes, from opening windows to saving files and submitting forms.

Java JButton

The JButton class is used to create a labeled button that has platform independent implementation. The application result in some action when the button is pushed. It inherits Abstract Button class.

2.2 What is event-driven programming?

Event-driven programming, or event-oriented programming, is a paradigm where entities (objects, services, and so on) communicate indirectly by sending messages to one another through an intermediary. The messages are typically stored in a queue before being handled by the consumers.

Event-driven programming also makes it easier to scale large systems, adding capacity simply by adding consumers.

Events are actions or occurrences that happen in the system. In the world of programming, HTML events are something that happens to the HTML elements. But when JavaScript is used in HTML pages, it can react to these events. Here are different types of events in JavaScript and how they work.

Types of Events in JavaScript

There are different types of events in JavaScript that are used to react to events. Here, some of the famous or most commonly used events such as:

OnClick, Onkeyup, Onmouseover, Onload, Onfocus, edureka

Components of Event Handling

Event handling has three main components,

1. Events: An event is a change in state of an object.
2. Events Source: Event source is an object that generates an event.
3. Listeners: A listener is an object that listens to the event. A listener gets notified when an event occurs.

How Events are handled?

A source generates an Event and send it to one or more listeners registered with the source. Once event is received by the listener, they process the event and then return. Events are supported by a number of Java packages, like java. util, java. awt and java. awt. event.

) Steps to handle events:

- Implement appropriate interface in the class.
- Register the component with the listener.

Placing controls on a frame

We can add and remove controls to a Container like Applet and Frame using the following methods available in the Container class:

Component add (Component ref)

Component remove (Component ref)

1_Label

2_Buttons

3_Checkboxes

4_Dropdown_Boxes

5 List Boxes

6 Text Fields

7_Text_Areas

Label

Buttons

A push button is the frequently found GUI control. A push button or a button can be created by using the *Button* class and its constructors which are given below:

`Button ()`

`Button (String str)`

Some of the methods available in the *Button* class are as follows:

void set Label (String str) – To set or assign the text to be displayed on the button.

String getLabel () – To retrieve the text on the button.

When a button is clicked, it generates an Action Event which can be handled using the Action Listener interface and the event handling method is `action Performed ()`. If there are multiple buttons we can get the label of the button which was clicked by using the method `get Action Command ()`.

3.1 Define classes with inheritance.

Classes are a template for creating objects? They encapsulate data with code to work on that data. Classes in JS are built on prototypes but also have some syntax and semantics that are unique to classes.

Class Inheritance

Inheritance enables you to define a class that takes all the functionality from a parent class and allows you to add more.

Using class inheritance, a class can inherit all the methods and properties of another class. Inheritance is a useful feature that allows code reusability.

Inheritance, Encapsulation and Polymorphism

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system). The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.

Why use inheritance in java

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Encapsulation

Encapsulation in Java is a *process of wrapping code and data together into a single unit*, for example, a capsule which is mixed of several medicines. We can create a fully encapsulated class in Java by making all the data members of the class private. Now we can use setter and getter methods to set and get the data in it. The **Java Bean** class is the example of a fully encapsulated class.

Advantage of Encapsulation

1. By providing only a setter or getter method, you can make the class **read-only or write-only**. In other words, you can skip the getter or setter methods.
2. It provides you the control over the data. Suppose you want to set the value of id which should be greater than 100 only, you can write the logic inside the setter method. You can write the logic not to store the negative numbers in the setter methods.
3. It is a way to achieve **data hiding** in Java because other class will not be able to access the data through the private data members.
4. The encapsulate class is **easy to test**. So, it is better for unit testing.
5. The standard IDE's are providing the facility to generate the getters and setters. So, it is **easy** and fast to create an encapsulated class in Java.

Polymorphism:

Polymorphism in Java is a concept by which we can perform a *single action in different ways*. Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So, polymorphism means many forms.

There are two types of polymorphism in Java: compile-time polymorphism and runtime polymorphism. We can perform polymorphism in java by method overloading and method overriding.

3.3 The rules of inheritance and accessibility.

Rules of Inheritance in Java

1. Extending a Class In Java, when a class extends another class, it means that the subclass inherits the properties and behaviors of the superclass. This establishes a relationship where the subclass is a more specialized form of the superclass.

2. Implementing an Interface When a class implements an interface in Java, it means that the class agrees to follow a certain contract defined by that interface. By implementing an interface, the class promises to provide the necessary functionality specified by the interface's methods.

3. Abstract Class Implementing an Interface Similar to a regular class, an abstract class in Java can also implement an interface. This allows the abstract class to define the required methods specified by the interface. However, since an abstract class cannot be directly instantiated, its concrete subclasses are responsible for providing the actual implementation of these methods.

4. Extending an Interface In Java, an interface can extend another interface, allowing it to inherit the methods of the parent interface while also introducing additional methods. This allows for the composition of interfaces and the specification of more specific behaviors.

5. Extending a Class and Implementing an Interface In Java, it is possible for a class to extend another class and implement one or more interfaces simultaneously. This allows the class to inherit the properties and behaviors of the superclass while also fulfilling the contracts defined by the implemented interfaces.

6. Limitations of Multiple Inheritance in Java In Java, multiple inheritance of state refers to the ability to directly inherit properties (fields) from multiple classes. Unlike some programming languages, Java does not support multiple inheritance of state. This decision was made to avoid conflicts and ambiguity that could arise from inheriting conflicting or duplicate properties.

7. Multiple Inheritance of Type in Java is allowed

Java supports multiple inheritance of type through interfaces. This means that a class can implement multiple interfaces, allowing it to inherit and exhibit multiple behaviors specified by those interfaces. By implementing multiple interfaces, a class can fulfill various contracts and have a more diverse range of functionality.

8. Inherited Access Modifiers in Java In Java, only the public and protected members (fields and methods) of a superclass are inherited by subclasses. These members are accessible and can be used within the subclass. Private members are not directly inherited and cannot be accessed by the subclass.

9. Constructors are not inherited In Java, constructors are not inherited by subclasses. Although the subclass can call the constructor of the superclass using the super keyword, the constructor itself is not inherited. The purpose of calling the superclass constructor is to initialize the inherited members of the superclass properly.

10. Type Reference and Declaration In Java, reference variables can be declared using the type of a superclass or interface, even if they refer to objects of the subclass or a class implementing that interface. This provides flexibility in handling objects and allows for polymorphism, where an object can be treated as an instance of its superclass or implemented interface.

For example, we can declare a reference variable of type Shape and assign it an object of the Circle class. This allows us to treat the circle object as a shape and call methods defined in the Shape interface, providing a more generic and flexible approach.

The rule of accessibility in Java

Refers to the ability of a Java program to be used by a wide range of users, including those with disabilities.

The different levels of accessibility in Java include:

Level A: This is the most basic level of accessibility, and includes features such as text alternatives for non-text content, keyboard accessibility, and compatibility with screen readers.

Level AA: This level builds on Level A, and includes additional features such as captions for pre-recorded audio content, and the ability to navigate content using a keyboard.

Level AAA: This is the most advanced level of accessibility, and includes features such as sign language interpretation for pre-recorded audio content, and the ability to navigate content using a keyboard and a screen reader.

It's important to note that accessibility is a continuous process, and there is no such thing as a completely accessible application. Developers should always strive to improve the accessibility of their applications, and to test them with real users who have disabilities.

3.4 How constructors of a class are affected by inheritance

Inheritance and Constructors

Constructors in Java are used to initialize the values of the attributes of the object serving the goal to bring Java closer to the real world. We already have a default constructor that is called automatically if no constructor is found in the code. But if we make any constructor say parameterized constructor in order to initialize some attributes then it must write down the default constructor because it now will be no more automatically called.

It is very important to understand how the constructors get executed in the inheritance concept. In the inheritance, the constructors never get inherited to any child class.

In java, the default constructor of a parent class called automatically by the constructor of its child class. That means when we create an object of the child class, the parent class constructor executed, followed by the child class constructor executed.

3.5 Concept of Abstract Super Class:

An Abstract Super Class is an abstract class which encapsulates the fields and methods that its sub types might need. It can also be used to provide default implementations of certain methods that can be overridden by sub types. It can also contain abstract instance methods which must be overridden by the sub types. Hence Abstract Super Class can be used to write high level abstract logic needed by sub types.

Characteristics of Abstract Super Class:

1. Ensures that logic common to its related classes are implemented consistently in one place.
2. Helps maintain the code reused in several classes in one place.
3. Helps sub types by implementing most of the required functionalities. Sub types only need to implement the abstract methods. Of course, the sub types are always allowed to improve the implementations of non-private instance methods through method overriding.

3.6 The process involved in writing programs using inheritance, encapsulation and polymorphism.

Each programming language has slightly different terminology for inheritance. In Java, the parent class is called the **superclass**, and the inheritor class is called the **subclass**. Developers may also call super class's base or parent classes and subclasses derived or child classes.

Subclasses are linked to super classes using the extends keyword during their definition.

4.1 Java Servlets

Java servlet is the *foundation* of the Java server-side technology, Java Server Pages (JSP), Java Server Faces (JSF), Struts, spring, Hibernate, and others, are extensions of the servlet technology.

Java servlets typically run on the HTTP protocol. HTTP is an *asymmetrical request-response protocol*. The client sends a *request message* to the server, and the server returns a *response message* as illustrated.

Servlets are server-side programs run inside a *Java-capable* HTTP server. Apache Tomcat Server is the official Reference Implementation (RI) for Java servlet and JSP, you need to install Tomcat to try out Java servlets.

4.3 Java Server Pages (JSP)

What is JSP?

Java Server Pages (JSP) is a technology that helps developers create dynamic, data-driven web pages. JSP pages are compiled into Java servlets and run on the server.

JSP uses a special syntax that embeds snippets of Java code within HTML, and these pages are stored as regular HTML files with a .jsp extension. This code is known as a JSP action.

JSP actions are executed when a user requests the page.

These pages are well suited for use in a distributed environment offering a high degree of flexibility.

JSP pages offer many benefits over traditional HTML pages, such as:

1. JSP pages can be easily created and maintained.
2. JSP is capable of producing dynamic content, but HTML is not.
3. JSP provides excellent performance and scalability.
4. The separation of visual logic and functionality makes it easier to manage.

Creating and mapping a Servlet

There are three ways to create the servlet.

1. By implementing the Servlet interface
2. By inheriting the Generic Servlet class
3. By inheriting the Http Servlet class

The Http Servlet class is widely used to create the servlet because it provides methods to handle http requests such as do Get (), do Post, do Head() etc.

In this example we are going to create a servlet that extends the Http Servlet class. In this example, we are inheriting the Http Servlet class and providing the implementation of the do Get () method. Notice that get request is the default request.

Servlet Mapping

Servlet mapping controls how you access a servlet. The following examples demonstrate how you can use servlet mapping in your Web application. In the examples, a set of servlet configurations and mappings (from the web.xml deployment descriptor) is followed by a table (see url-patterns and Servlet Invocation) showing the URLs used to invoke these servlets.

Mapping a Servlet with the web XML file

The Servlet life cycle will be managed by the Servlet container that is under the Web/Application server. Whenever an Http Request comes from the client browser, the servlet container will map the request to the corresponding servlet based on the URL mappings provided in the deployment descriptor file – web.xml. **For example:** Consider the below servlet mappings in the web.xml file.

XML

```
< servlet >

  < servlet-name > Hello Servlet < / servlet-name >

  < servlet-class > Hello Servlet < / servlet-class >

< /servlet >

< servlet-mapping >

  < servlet-name > Hello Servlet < / servlet-name >

  < url-pattern > / hello < / url-pattern >

< /servlet-mapping >
```

Servlet Annotations

Java Servlets provides a number of annotations to allow users to declare servlets, filters, and listeners in the javax. Servlet annotation package. It also provides annotations to specify metadata for the components that are declared.

Servlet with Annotation Examples

For better understanding, we will create a simple HTML page to map the Servlet with the URL.

Example 1:

Servlet annotated with only URL pattern:

index.html

HTML

```
< !DOCTYPE html >

< html >

< head >

< meta charset = " ISO – 8859 - 1" >

< title > Home < / title >

< / head >

< body >

    < form action = "hello" method = "post" >

        Welcome Page: < input type =" submit " / >

    < / form >

< / body >

< / html >
```

Servlet Request Interface

An object of Servlet Request is used to provide the client request information to a servlet such as content type, content length, parameter names and values, header information's, attributes etc.

What is HTTP?

The Hypertext Transfer Protocol (HTTP) is designed to enable communications between clients and servers. HTTP works as a request-response protocol between a client and server.

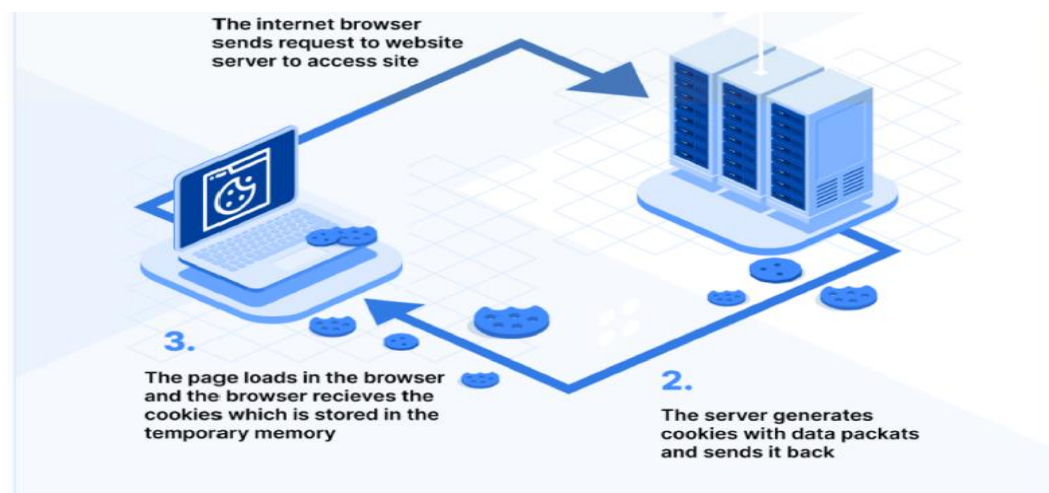
Example: A client (browser) sends a HTTP request to the server; then the server returns a response to the client. The response contains status information about the request and may also contain the requested content

4.4 What are session cookies?

Session cookies are cookies that last for a session. A session starts when you launch a website or web app and ends when you leave the website or close your browser window. Session cookies contain information that is stored in a temporary memory location which is deleted after the session ends. Unlike other cookies, session cookies are never stored on your device. Therefore, they are also known as transient cookies, non-persistent cookies, or temporary cookies.

How do session cookies work?

The session cookie is a server-specific cookie that cannot be passed to any machine other than the one that generated the cookie. The server creates a “session ID” which is a randomly generated number that temporarily stores the session cookie. This cookie stores information such as the user’s input and tracks the movements of the user within the website. There is no other information *stored in the session cookie*.



Session cookies allow websites to remember users within a website when they move between web pages. These cookies tell the server what pages to show the user so the user doesn't have to functionalities depend on users' activities.

What are Cookies?

Cookies provide a way for web applications to store information in the user’s browser. This information can be retrieved every time the user requests a page from the same web server that created the cookies.

Cookies can be either first-party or third-party. First-party cookies are created by the website that the user is visiting, while third-party cookies are created by domains other than the website being visited. Third-party cookies are often used for advertising and tracking purposes.

High-level of how cookies work

First, a cookie is a text string stored by the browser as a key/value pair.

```
PHPSESSID=D1F15245171203E8670487F020544490
user_id=87
email=jsmith@hotmail.com
userName=jsmith
passwordCookie=opensesame
```

Fig 1. Example of cookies

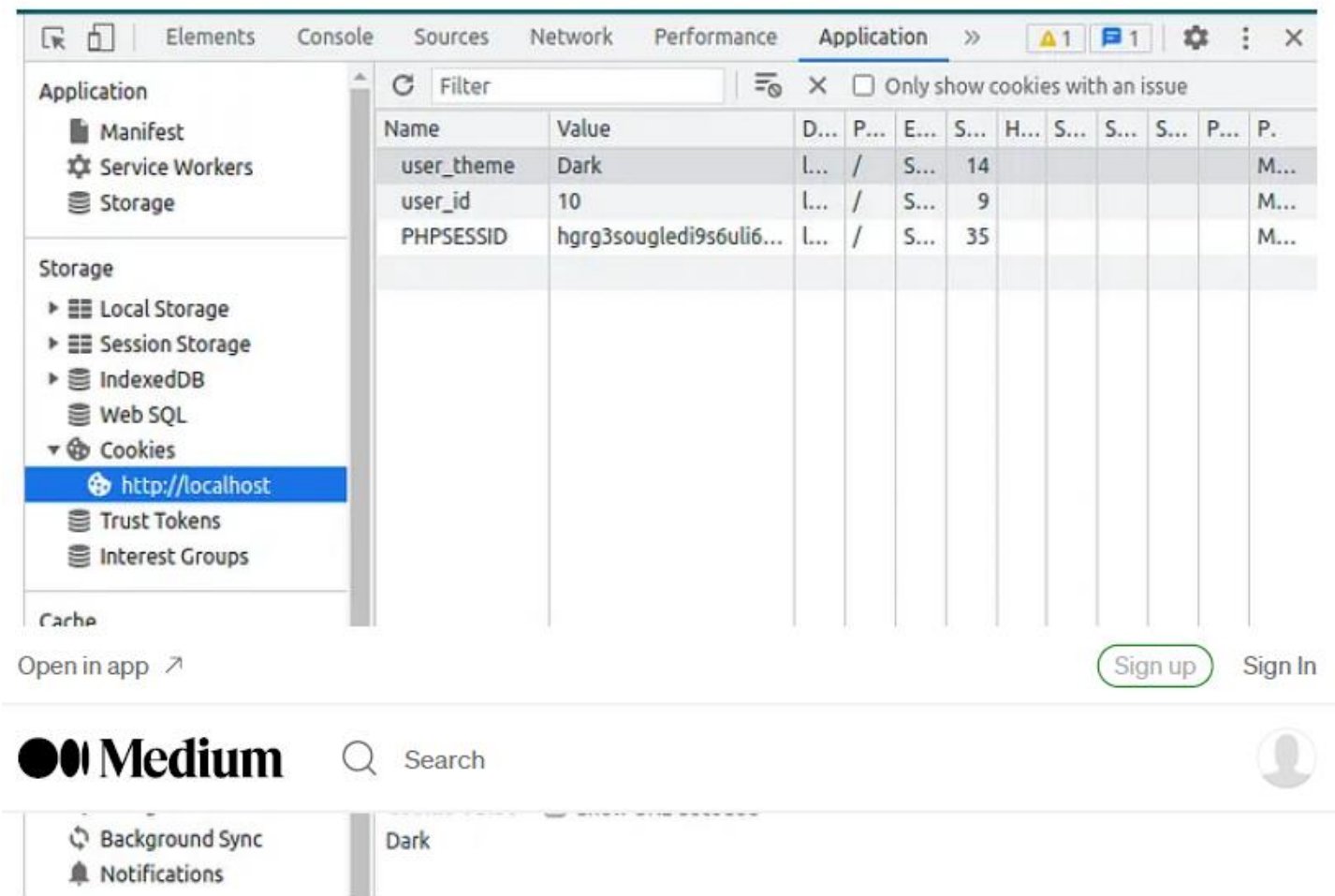


Figure 2. Examples of cookies in the browser.

What are Sessions?

Sessions are a way of storing information about a user on the server-side. Those information's will then be used in subsequent requests.

By definition, a Web session is a sequence of network HTTP request and response transactions associated with the same user.

Sessions provides the ability to establish variables — such as access rights and localization settings — which will apply to each and every interaction a user has with the web application for the duration of the session.

How session work?

When a user logs into the website, a session is created. In this session, you can created variable called “session variable” that store data in a key/value format (like cookies).

This session is associated with a randomly generated unique ID, which is created by the server. It's called “session ID”.

The generated session ID is then sent to the user's browser and stored as a cookie, while the session data is stored on the server-side.

Now, when the browser send a request to the server, it'll send the cookies with the request.

The server will receive the cookie from the incoming request and retrieve the value of the session ID.



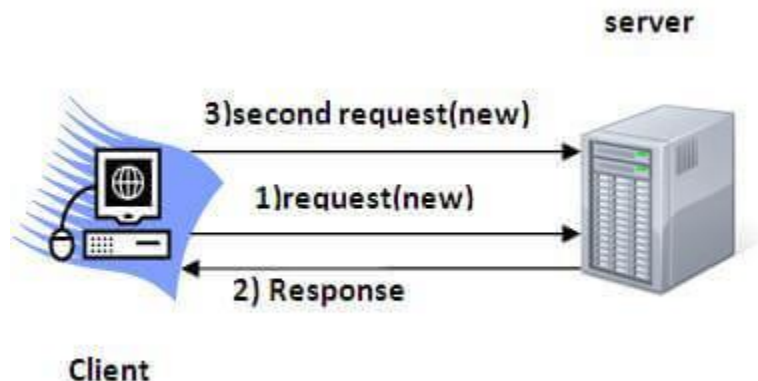
Session and tracking

Session simply means a particular interval of time

Session Tracking is a way to maintain state (data) of a user. It is also known as session management in servlet.

Http protocol is a stateless so we need to maintain state using session tracking techniques. Each time user requests to the server, server treats the request as the new request. So we need to maintain the state of an user to recognize to particular user.

HTTP is stateless that means each request is considered as the new request. It is shown in the figure bellow



Why use Session Tracking?

To recognize the user, it is used to recognize the particular user.

Session Tracking Techniques

There are four techniques used in Session tracking:

1. Cookies
2. Hidden Form Field
3. URL Rewriting
4. Http Session

4.5 Expression Language (EL)

The **Expression Language (EL)** simplifies the accessibility of data stored in the Java Bean component, and other objects like request, session, application etc. There are many implicit objects, operators and reserve words in EL. It is the newly added feature in JSP technology version 2.0.

Syntax for Expression Language (EL)

`${expression}`

4.6 Developing JSP

JSP technology is used to create web application just like Servlet technology. It can be thought of as an extension to Servlet because it provides more functionality than servlet such as expression language, JSTL, etc. A JSP page consists of HTML tags and JSP tags. The JSP pages are easier to maintain than Servlet because we can separate designing and development. It provides some additional features such as Expression Language, Custom Tags, etc.

Creating a simple JSP Page To create the first JSP page, write some HTML code as given below, and save it by .jsp extension. We have saved this file as index. jsp. Put it in a folder and paste the folder in the web-apps directory in apache tomcat to run the JSP page.

index. JSP

Let's see the simple example of JSP where we are using the scriptlet tag to put Java code in the JSP page.

```
< html >
< body >
< % out. Print (2*5); % >
< / body >
< / html >
```

It will print **10** on the browser.

How to run a simple JSP Page?

Follow the following steps to execute this JSP page:

1. Start the server
2. Put the JSP file in a folder and deploy on the server
3. Visit the browser by the URL `http://localhost:portno/contextRoot/jspfile`, for example, `http://localhost:8888/myapplication/index.jsp`

Coding EL and JSTL

The JSTL EL or JSTL Expression Language is used by JSP programmers in order to avoid the usage of Java Code for accessing data. The EL statements are always used within { } along with the prefix \$. Here is a simple example in which EL expression is used on the JSP page. The HTML page that contains the user name is first shown, and the entered username is then presented using a JSP page.

Source Code

create an input.html page.

```
< ! DOCTYPE html >
< html >
< head >
  < title > Form Example < / title >
< / head >
< body >
  < form method = "post" action = "demoel. jsp" >
    < input type = "text" name="ashish _ text" >
    < input type = "submit" value = "Submit" >
  < / form >
< / body >
< / html >
```

Coding JavaBean

Introduction

Bean refers to a component. Components are items that can be reused. A reusable component is referred to as a JavaBean in JSP. A Java Bean is a standard Java class with attributes, setters, and getters to represent a specific user form on the server. A JavaBean in JSP is a Java class that follows a set of guidelines. The following rules apply:

1. The class must be public, not final, and abstract.
2. It is strongly suggested that serializable be implemented.
3. A direct/indirect param constructor is required.
4. Non-static and private member variables should be used.
5. Every bean property should have one setter and getter method with the public modifier.

}

Using standard JSP tags with JavaBeans

JSP: use Bean action tag

The JSP: use Bean action tag is used to locate or instantiate a bean class. If bean object of the Bean class is already created, it doesn't create the bean depending on the scope. But if object of bean is not created, it instantiates the bean.

Syntax of JSP: use Bean action tag

1. < JSP: use Bean id = "instanceName" scope= "page | request | session | application"
2. **Class**= "package Name. Class Name" type= "package Name. class Name"
3. Bean Name="package Name. Class Name | <%= expression >" >

4. </JSP: use Bean > .

5.1 Database access with JDBC

Java Database Connectivity (JDBC) is the Java API that manages connecting to a database, issuing queries and commands, and handling result sets obtained from the database. JDBC was one of the earliest libraries developed for the Java language.

JDBC was initially conceived as a client-side API, enabling a Java client to interact with a data source. That changed with JDBC 2.0, which included an optional package supporting server-side JDBC connections. Every new JDBC release since then has featured updates to both the client-side package (java. SQL) and the server-side package (javax. SQL). JDBC 4.3, the most current version as of this writing, was released as part of Java SE 9 in September 2017 as JSR 221.

How JDBC works

As a developer, you can use JDBC to interact with a database from within a Java program. JDBC acts as a bridge from your code to the database, as shown in Figure 1.

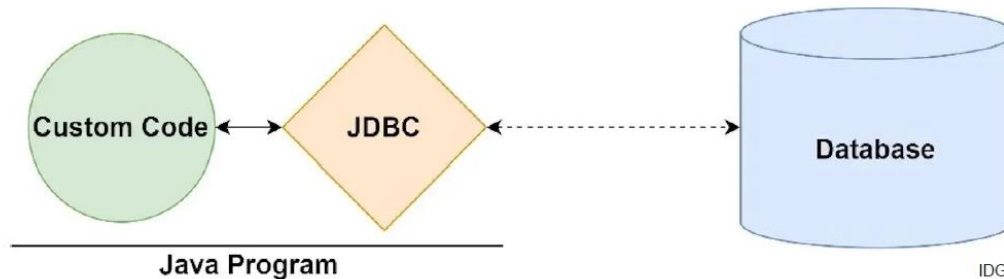


Figure 1. JDBC connects Java programs to databases.

What is a web application?

A web application (web app) is an application program that is stored on a remote server and delivered over the internet through a browser interface. Web services are web apps by definition and many, although not all, websites contain web apps.

Developers design web applications for a wide variety of uses and users, from an organization to an individual for numerous reasons. Commonly used web applications can include webmail, online calculators or e-commerce shops. While users can only access some web apps by a specific browser, most are available no matter the browser.

How does web applications work?

Web applications do not need to be downloaded since they are accessed through a network. Users can access a web application through a web browser, such as Google Chrome, Mozilla Firefox or Safari.

For a web app to operate, it needs a web server, application server and database. Web servers manage the requests that come from a client, while the application server completes the requested task. A database stores any necessary information.

Web applications typically have short development cycles and small development teams. Developers write most web apps in JavaScript, HTML5 or CSS. Client-side programming typically utilizes these languages, which help build an application's front-end. Server-side programming creates the scripts a web app will use. Languages such as Python, Java and Ruby are commonly used in server-side programming.

Web Application Development Process

The web application development process involves several phases, each with its own set of tasks, activities, and considerations. While the specific steps may vary depending on the project scope, technology stack, and team structure, here is a general outline of the web application development process:

1. Requirement Gathering and Analysis:

- Understand the project's goals, target audience, and business requirements.
- Collaborate with stakeholders to gather detailed project requirements and expectations.
- Define functional and non-functional requirements for the web application.

2. Planning:

- Create a project plan that outlines the scope, timeline, budget, resources, and milestones.
- Determine the technology stack, frameworks, and tools that will be used.
- Plan the architecture of the application, including data models, components, and interactions.

3. Design:

- Design the user interface (UI) and user experience (UX) of the web application.
- Create wireframes, prototypes, and mockups to visualize the layout and interactions.
- Design database schemas, data flows, and system diagrams.

4. Front-End Development:

- Develop the client-side of the web application using HTML, CSS, and JavaScript.
- Implement the UI design, including responsive layouts and interactive elements.
- Integrate front-end frameworks and libraries as needed.

5. Back-End Development:

- Develop the server-side logic, business logic, and APIs using the chosen programming language and framework (e.g., Python with Django or Flask, Node.js with Express, Ruby on Rails, etc.).
- Implement user authentication, authorization, and security measures.
- Create RESTful or GraphQL APIs for communication between front-end and back-end.

6. Database Development:

- Design and create the database schema based on the application's data requirements.
- Choose an appropriate database management system (e.g., MySQL, PostgreSQL, MongoDB) and set up the database.
- Implement data storage, retrieval, and manipulation operations.

7. Integration:

- Integrate third-party services, APIs, and libraries that enhance the application's functionality (e.g., payment gateways, social media integration, geolocation services).

8. Testing:

- Conduct various levels of testing, including unit testing, integration testing, and user acceptance testing.
- Test the application's functionality, performance, security, and compatibility across different browsers and devices.
- Identify and fix bugs, errors, and inconsistencies.

9. Deployment:

- Deploy the web application to a production environment, which could be a web server, cloud platform, or hosting service.
- Configure server settings, domain name, and security measures (SSL/TLS certificates).
- Set up monitoring and error tracking tools to ensure the application's stability.

10. Maintenance and Updates:

- Continuously monitor the application's performance, security, and user feedback.
- Regularly update and maintain the application, including bug fixes, security patches, and feature enhancements.
- Scale the application as needed to accommodate increased user demand and traffic.

11. Documentation and Training:

- Create user documentation, including user guides and tutorials, to help users navigate and use the application effectively.
- Provide training and support to users, administrators, and other stakeholders as required.

12. Post-Launch Activities:

- Market and promote the web application to attract users and drive engagement.

- Gather user feedback and analyze metrics to make informed decisions for future updates and improvements. Throughout the development process, collaboration, communication, and iterative refinement are essential to ensure the successful creation of a functional, user-friendly, and reliable web application that meets the needs of both users and stakeholders.

Developing mobile applications using Java Script

In recent years, there has been a surge in the use of JavaScript for mobile app development. This is because JavaScript offers many benefits compared to other programming languages.

Some of the benefits of using JavaScript for mobile app development include:

1. Ease of Use: JavaScript

Ease of Use: JavaScript is easy to learn and use. This makes it ideal for those who are new to programming or do not have much experience with coding.

2. Flexibility: JavaScript

Flexibility: JavaScript is very flexible and can be used to create a wide range of apps.

3. Speed: Js apps

Speed: Js apps are generally faster than those created with other programming languages.

4. Cross-Platform Compatibility

Cross-Platform Compatibility: JavaScript apps can be used on both Android and iOS devices.

Development Environment Setup

The first step in developing a JavaScript mobile app is to set up your development environment. For this, you will need a text editor and a web browser. There are many different text editors available, but we recommend using Visual Studio Code or Sublime Text.

Once you have installed a text editor, you will need to install the following software:

Node.js

1. Node.js: This is a server-side scripting platform that enables you to run JavaScript code outside of a web browser.

React Native

2. React Native: This is a framework that allows you to create cross-platform mobile apps using JavaScript.

Expo

3. Expo: This is a tool chain that makes it easy to develop and publish React Native apps.

To set up your development environment, you will need to create a new project directory. Open your text editor and create a new file called `index.js` in this directory. Add the following code to this file:

```
console.log ("Hello, world!");
```

Save the file and open it in your web browser. You should see the message "Hello, world!" printed in the console.

Process of Creating a Simple "To-Do" App

Now that you have set up your development environment, you are ready to start coding your first app. In this section, we will walk you through the process of creating a simple "To-Do" app. This app will allow users to add tasks to a list and mark them as complete when they are finished.

1. Create a new file called "todo.js" in your project directory. Add the following code to this file:

```
const todos = [];
```

2. Next, you will need to create a function that will add new tasks to the to-dos array. Add the following code to the "todo.js" file:

```
function add To Do (task) {  
  todos.push (task);  
}
```

3. Now, you will need to create a function that will mark tasks as complete. Add the following code to the `todo.js` file:

```
function complete To Do (index) {  
  todos.splice (index, 1);  
}
```

4. Finally, you will need to create a function that will print all of the tasks in the to-dos array. Add the following code to the `todo.js` file:

```
function print To Dos () {  
  console.log (to dos);  
}
```

5. Save the file and open it in your web browser. You should see the message "Hello, world!" printed in the console.

5.3 The basic concept of programming using JavaScript

Irrespective of the programming language you choose to learn, the basic concepts of programming are similar across languages. Some of these concepts include: Variable declaration, Basic syntax, Data type and structures, Flow control structures (Conditionals and loops), Functional programming, Object-oriented programming, and Debugging.

Variable declaration

Variables are containers for storing data values, a memory location for a data type. Variables are created using a declaration or keyword that varies across languages.

Variable names are usually alphanumeric, that is, they contain a-z and 0-9. They can also include special characters like underscore or the dollar sign.

Variables can hold values of any data type supported by the programming language. This value may change during program execution.

Data types and structures

Data types refer to the classification of data. The most common data types include:

String

Boolean (true or false)

Numbers, which includes integers (whole numbers from 1) and floating-point numbers (decimal-base)

Characters (includes single alphabets or numbers)

Arrays (a collection of data, usually of the same data type)

A **Data Structure** is a collection of data values. These structures include operations that can be applied to that data. Data structures are important in computer programming for organizing, managing, and storing data quickly and efficiently.

Some common types of data structures include: Stacks, Heaps, Trees, Linked lists, Queues, Arrays, Tables, and Graphs

Flow control structures

Flow Control Structures are the fundamental components of computer programs. They are commands that allow a program to “decide” to take one direction or another.

There are three basic types of control structures: sequential, selection, and iteration.

Sequential

Object-Oriented Programming (OOP) is a programming concept that revolves around ‘objects’ and ‘methods.

There are four principles of OOP: Inheritance, Polymorphism, Abstraction, and Encapsulation.

5.4 Embedding JavaScript in HTML directly

You can choose between two different methods to include JavaScript. We'll introduce you to both.

To load JavaScript elements as quickly as possible, place the script element in the HTML head or body. In new browsers, a relatively simple JS source code is sufficient. In the following example, the message "Hello friend" is written on a website via JavaScript integration:

```
<!DOCTYPE html >

<html >

<head >

<Title > JavaScript: Hello Friend</title>

<Script >
Alert ("Hello Friend! ");
</script >
</head>

<Body >

<p > this website only displays a message box. </p >

</body >

</html>
```

Program control logic

Control logic is a key part of a software program that controls the operations of the program. The control logic responds to commands from the user, and it also acts on its own to perform automated tasks that have been structured into the program

The control logic operates in binary, which means that software users cannot communicate with it directly. Thus, software programs have to be equipped with a feature that can translate commands from the user into binary so the control logic can understand them. The output of the control logic occurs in binary, but is translated into a function or message by the program.

A programmable logic controller (PLC) is a specialized Programmable device which is used to control machines and processes. It uses a programmable memory to store instructions and execute specific functions that include on/off control, timing, counting, sequencing, arithmetic, and data handling.

Advantages

1. Increased Reliability

2. More Flexibility
3. Lower Cost
4. Communications Capability
5. Faster Response Time
6. Easier to Troubleshoot

What is an enterprise solution?

An enterprise solution is a software application designed to meet the multifaceted needs of an organization rather than an individual end user or small business. Being large Organizations with broad ranges or operations, enterprises often face unique challenges, such as interdepartmental communication and international commerce. The use of enterprise solutions can help an enterprise to overcome these challenges, thus improving its performance. Enterprise solutions commonly integrate multiple systems within an organization, allowing for the correlation or dissemination of information across its various databases, which can streamline business processes and improve workflows. They can be particularly useful for enterprise professionals whose responsibilities cover multiple functional areas within the organization. A compliance officer, for example, may require access to the systems of all the departments of an enterprise.

Application areas of enterprise solutions

There are multiple types of software solutions intended for enterprises or that may be useful to them. Some of the most broadly used enterprise solutions include:

Accounting

An accounting solution provides tools that facilitate the completion of tasks relating to financial transactions and management. At the enterprise level, a common challenge in the area of finances is the access and integration of monetary data from various functional areas.

Accounting enterprise solutions commonly aim to resolve this challenge by centralizing financial data and providing automation capabilities for repeatable processes. They also commonly include features that allow enterprises to maintain a general ledger, complete tax reporting obligations, track accounts and assets, manage the payroll and handle billing concerns.

Business intelligence refers to the concept and practice of drawing insight from business data and using it to inform sound decisions. A business intelligence solution facilitates the practice by integrating data from various sources and visualizing it, allowing analysts a clearer view of the enterprise's performance and areas of improvement. Such software commonly draws data from sales, marketing and manufacturing databases, has customizable dashboard capabilities and features robust reporting functions.

Customer relationship management

Customer relationship management, commonly known by the initialism CRM, is a category of tools for tracking customer data that can help an organization to improve their relationships with their customers and thus increase

revenue. An effective CRM solution can help an organization to create better sales techniques and streamline the stages of CRM, from developing leads to retaining existing customers.

What is Software Development Lifecycle (SDLC)?

Software Development Lifecycle is a systematic series of steps to develop and test high-quality software. SDLC allows developers to not only approach software development in a systematic way, but it also allows them to deliver it on time and in the budget. SDLC is a development tracking mechanism, broken into seven steps that are to be carried out in a particular series. Following it makes it easier for the developers to plan the project and offer near-accurate time and cost estimations. SDLC helps in speedy and cost-effective software development, and by increasing visibility and transparency, it helps developers increasing better relations with their customers and decreases project risk.

The process of developing enterprise solution

Developing software is not a set-in stone method. There is no universal approach to it or the right way to do it. Sometimes a development process fits the software that you are trying to make like skin, but sometimes you have to go for unconventional methods. It all depends on your needs and the type of software that you are developing.

Over the years, enterprise software development has evolved drastically; new methods have surfaced, making the process more streamlined, transparent, and efficient, and older workflows have been discarded. But again, it is an iterative process, and every few years a new development process becomes the hot trend. (<https://www.matellio.com>)

And while there is no universally accepted approach to develop software, we do, however, have Software Development Lifecycle (SDLC) - a seven-step software development approach to build, test, and deliver quality software products. While SDLC is widely considered as the best approach to enterprise software development still, it is not a necessity that you use it to develop your software as well.

7-Steps to Develop an Enterprise Software

SDLC is broken into seven steps that are to be carried out in the sequence that has been laid down below. It is to be kept in mind that following this sequence will provide optimal outcomes while deviating from it might not have the desired result.

Step 1: Requirement Analysis

Step 2: Feasibility Study

Step 3: Design

Step 4: Coding

Step 5: Testing

Step 6: Install/Deploy

Step 7: Maintenance

1. Requirement Analysis

Requirement Analysis is the first step in the SDLC process. It is where the board members and stakeholders get together to devise a plan of action and decide what exactly is wanted out of the software. This is where all the operations that are required from the software are laid down.

2. Feasibility Study

This is the second step in SDLC, and the deliverables from the first step are used in this step to create documents. This is also the step where the minor details about the project are finalized as well. And once that is done, all of this data is documented so that everyone knows the plan, and there is no barrier of knowledge.

Design

The output of the previous phase, the SRS document, is now used to design the software. Different modules are built and documented, software architecture is developed according to the scope of the project. This is where the nooks and crannies of a project are defined and designed, from the smallest of modules to the overall architecture of the software.

4. Coding

This is the longest and most difficult phase of the project. Coding is the step in which the deliverables of the previous step are used, and developers start to actually create them, to code them in the language that was decided in the first step.

Testing

Once the developed software is ready, it is deployed to the testing environment where a team of QA engineers works with it. Their prime task is to find flaws, glitches, inconsistencies, and bugs, functional or UI/UX, in your software.

Code Quality: This testing phase focuses on the coding of each module to make sure that it is easily understandable. When the project is handed over to the end client, chances are he will have his coders poke around in the software as well, or maybe a new coder might be assigned to the project, and he will require a clean code to understand the project.

Unit Testing: In this testing, individual units of the software are tested by the QA engineers. The idea behind this testing is to make sure that each component of the software works the way it was designed to work.

Integration Testing: In this testing, the integrations of different software modules are tested. Since different modules were developed by different coders and are then put together, sometimes, it is possible that a tester might pick a bug in which two modules might not be interacting with each other the way they were supposed to. That is why Integration Testing is essential.

Performance Testing: In this testing, the performance of the software is scrutinized. The QA engineers take a look at the software's response time, speed, resource utilization, scalability, etc. They also make sure that the

software works flawlessly and holds up to certain performance standards even when more than one of its instances are active.

Security Testing: In this testing, the security of the software is checked. Any backdoor that might have been left open, any threat, vulnerabilities, etc. are all checked in this testing.

6. Install/Deploy

Once the software is tested and free of bugs, it is ready to be deployed and make its way to the stakeholders. If the project manager gives the go-ahead once all the bugs have been fixed, the software is deployed. Some companies also make use of automated deployment tools as well.

7. Maintenance

Once the software has been deployed, it is time to keep it running and upgrade it according to your client's needs or your stakeholder's needs. Some new additions to the software, some tweaks in the UI, all of it after the deployment process falls into this step. Regular upgrades and security fixes for the latest cyber threats are also necessary to keep the software safe from outside attacks.

Enterprise Application Solutions

Enterprise Application are sets of software applications that companies use to enhance the operation and business efficiency of their organizations.

Some of the business areas that these enterprise solutions address could be in the areas of Enterprise Resource Planning (ERP), Sales Funnel Management, Customer Relationship Management CRM), Human Resource Management, Approval Flow Process Management, Document Control, Business Intelligence, Messaging and many others.

Java AWT (Abstract Window Toolkit)